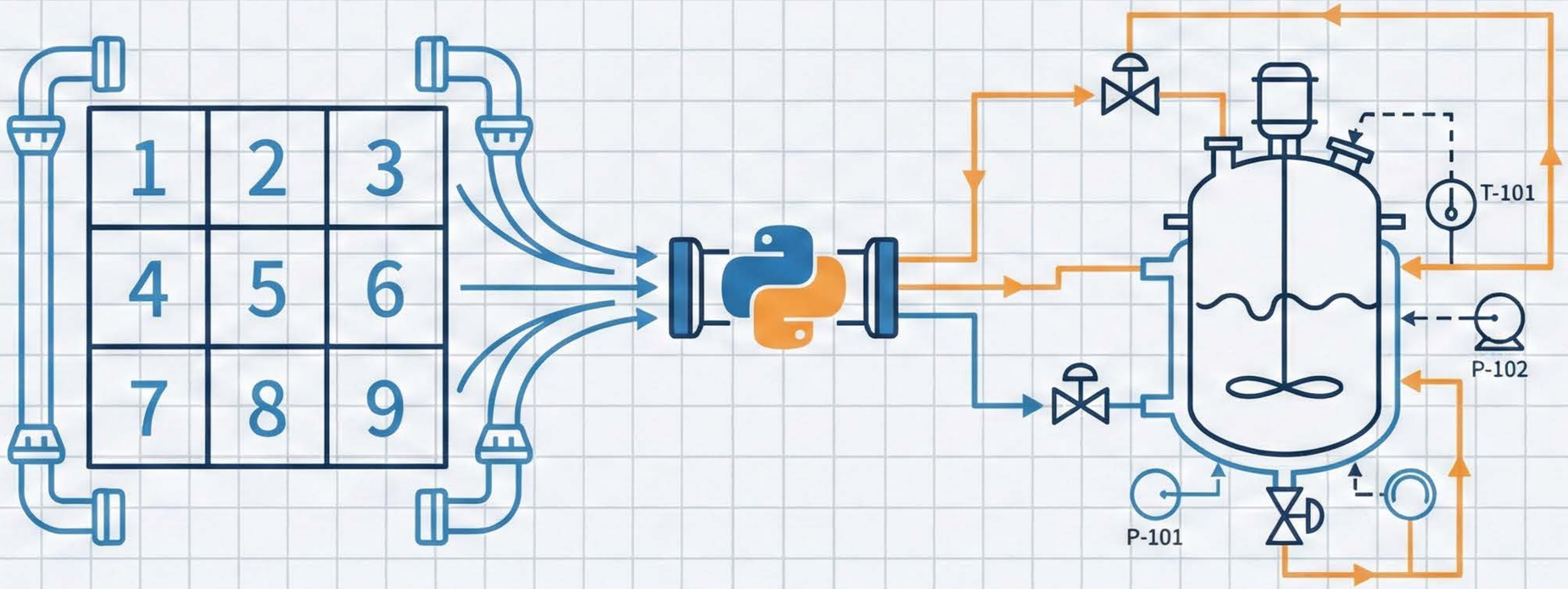
7. 線性代數運算

Unit 03 NumPy 數值計算基礎



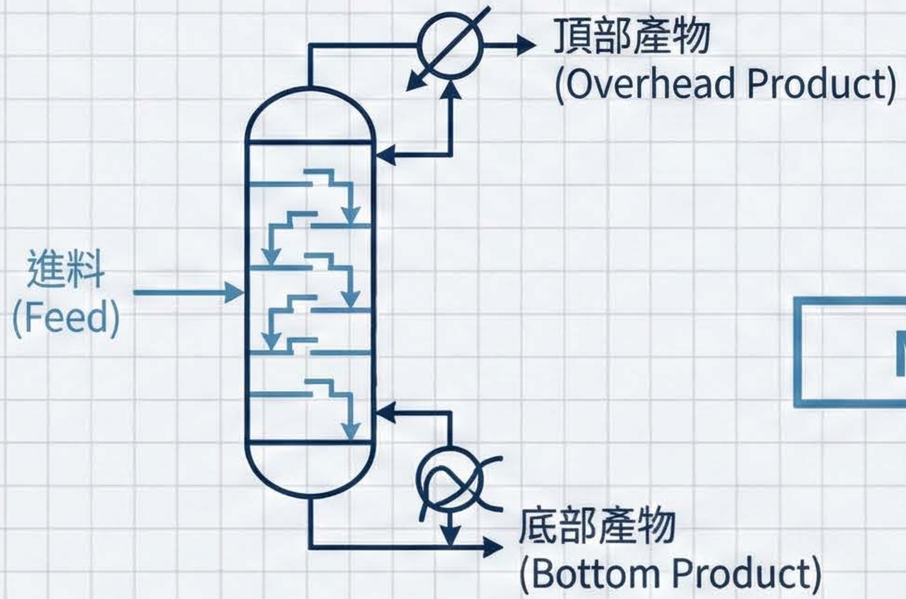
構建化工程序模擬的數學引擎

| AI in Chemical Engineering Application

NotebookLM

為什麼線性代數是化工模擬的核心？

物理系統 (Physical System)



- 物料平衡 (Material Balance)
- 能量平衡 (Energy Balance)
- 反應計量 (Reaction Stoichiometry)

Modeling

數學模型 (Mathematical Model)

$$Ax = b$$

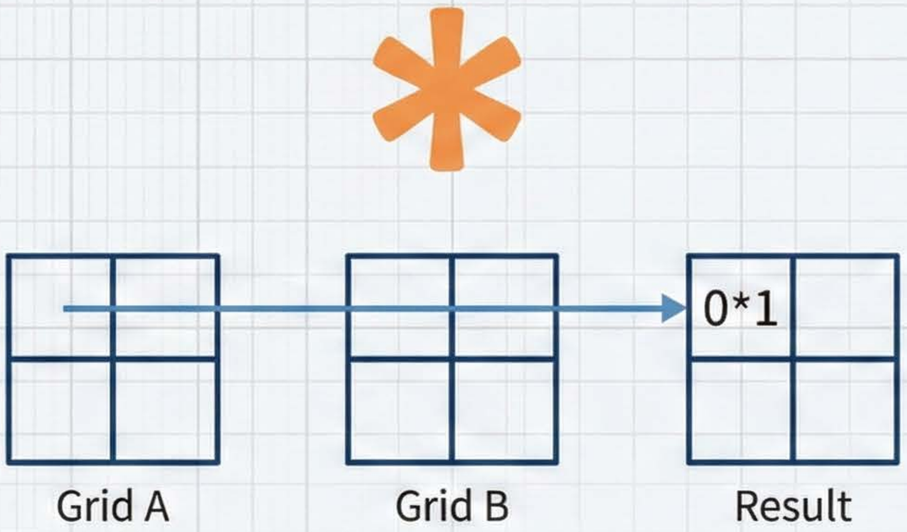
- **A**: 系統屬性矩陣 (System Matrix)
- **x**: 未知變數向量 (Unknowns)
- **b**: 輸入/邊界條件 (Input Vector)

NumPy 的線性代數模組 (`numpy.linalg`) 提供與 MATLAB 同級的高效能運算能力，是解決複雜化化工問題的基石。

NotebookLM

7.1 矩陣乘法：理解運算機制

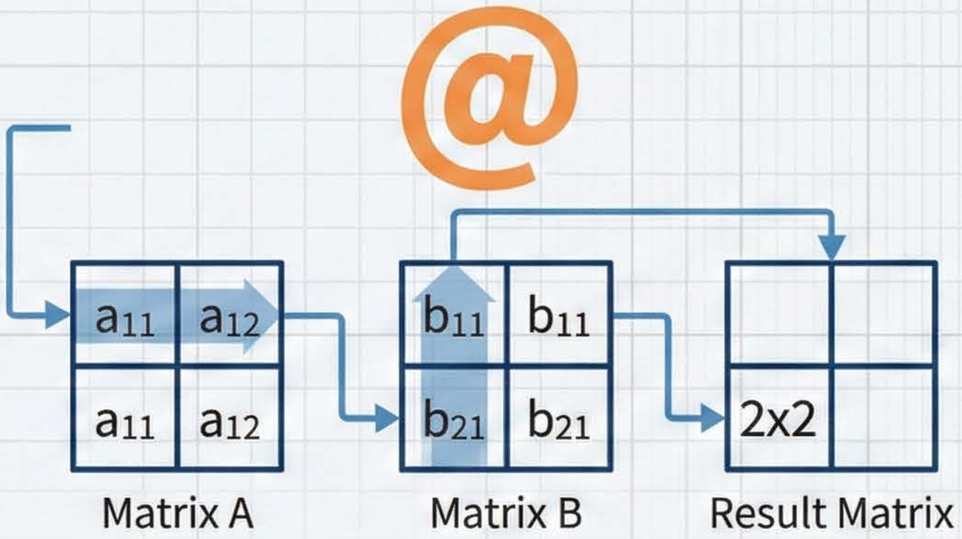
元素級乘法 (Element-wise)




```
>_  A * B
```

對應位置元素相乘

矩陣乘積 (Dot Product)



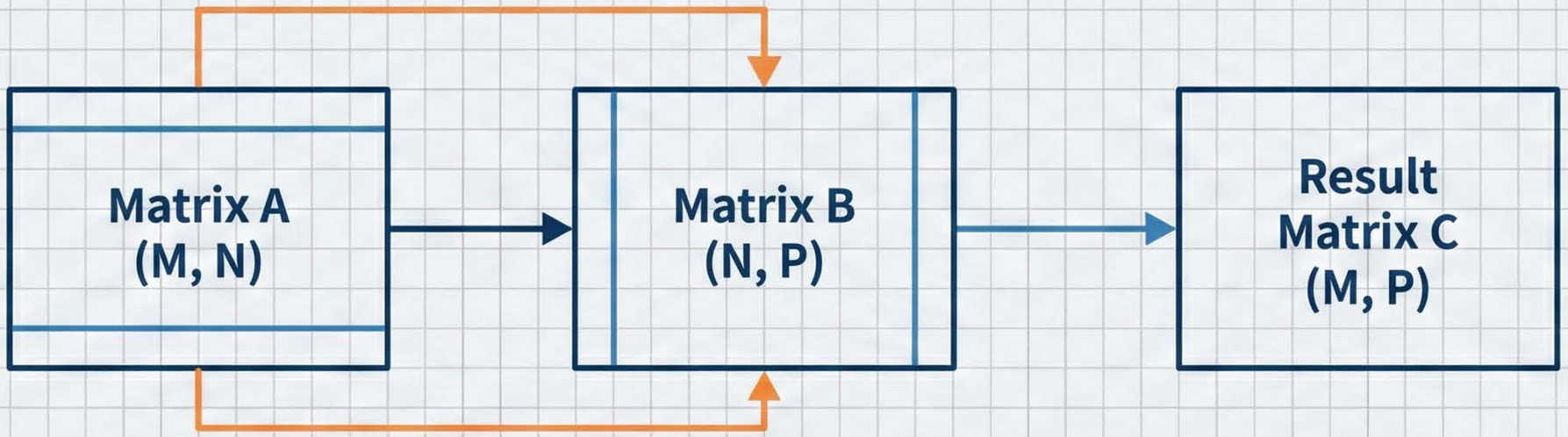

```
>_  A @ B
```

線性代數標準乘法

⚠ 注意：化工計算（如混合、反應）通常使用矩陣乘積 @，而非元素乘法。

NotebookLM

維度規則：確保數據流對接

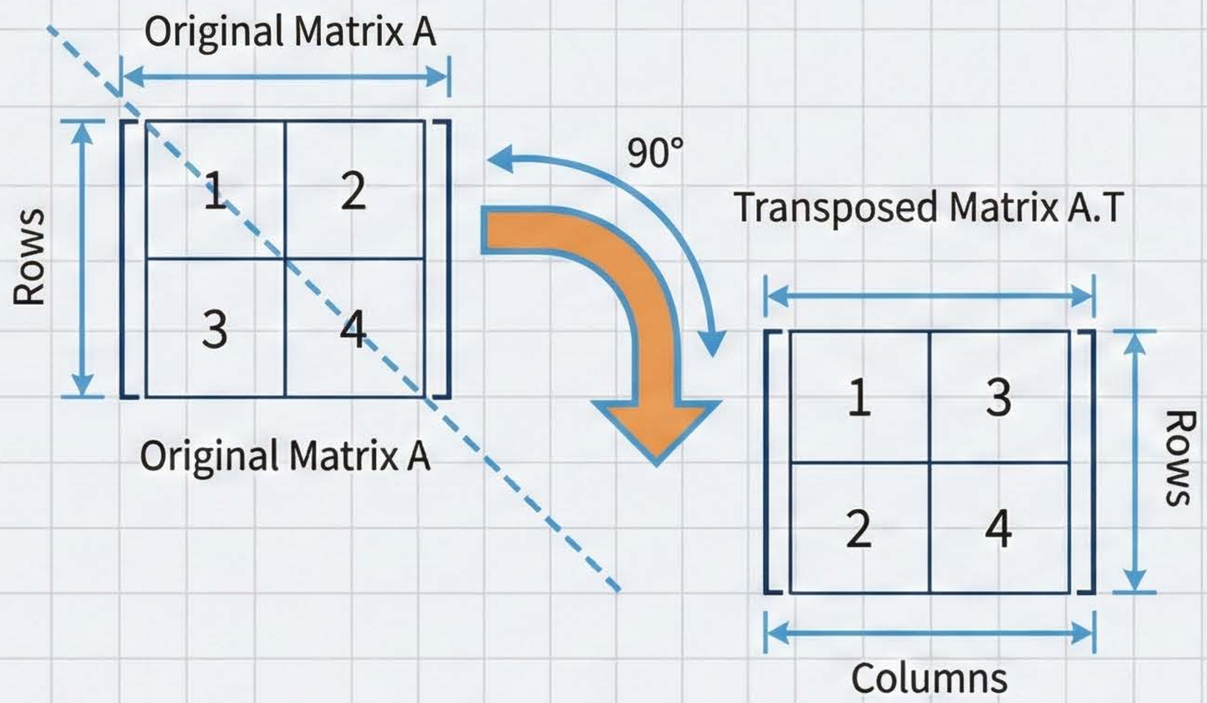



```
import numpy as np

A = np.random.rand(3, 4)  # 3 rows, 4 columns
B = np.random.rand(4, 2)  # 4 rows, 2 columns

C = A @ B    # Result shape: (3, 2)
# C = B @ A  # Error! (4,2) cannot multiply (3,4)
```

⚠ 矩陣乘法不具有交換律。前者的行數 (Columns) 必須等於後者的列數 (Rows)。

7.2 矩陣基礎運算：轉置 (Transpose)

改變數據的觀察視角



語法 (Syntax)

- 屬性 (Attribute)：arr.T (最常用)
- 函式 (Function)：np.transpose(arr)

```
arr = np.array([[1, 2], [3, 4]])
print(arr.T)

# Output:
# [[1, 3]
#  [2, 4]]
```

在處理多變數數據或準備繪圖資料時，常需將行向量轉置為列向量以符合廣播規則。

NotebookLM

7.2 矩陣進階運算：逆矩陣與行列式

逆矩陣 (Inverse Matrix)

$$A \cdot A^{-1} = I$$

```
np.linalg.inv(A)
```

僅適用於方陣 (Square Matrix only)

行列式 (Determinant)

```
np.linalg.det(A)
```

det(A)

== 0 → 奇異矩陣 (Singular) 無唯一解

!= 0 → 可逆 (Invertible) 系統可解

7.3 解線性方程組：化工計算的核心

$$Ax = b$$

The Bad Way

程式碼 (Code)

```
x = np.linalg.inv(A) @ b
```

數值穩定性差，運算速度慢

The Best Practice

程式碼 (Code)

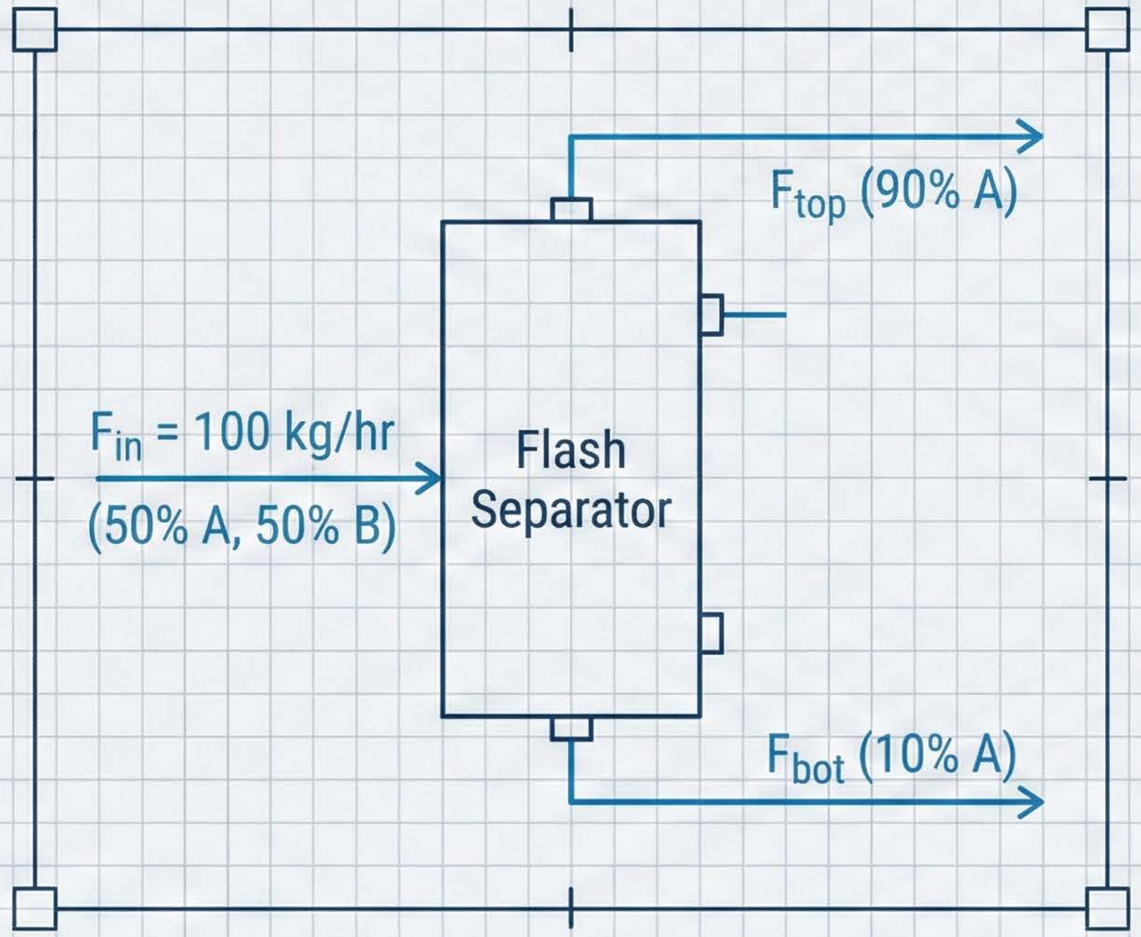
```
x = np.linalg.solve(A, b)
```

使用優化的分解演算法 (LU Decomposition)，效率更高，工業標準。

已知系統參數 **A** 與結果 **b**，求解未知變數 **x**。

NotebookLM

實務應用：單元操作物料平衡



質量平衡方程式 (Mass Balance Equations)

1. 總質量: $1 \cdot F_{top} + 1 \cdot F_{bot} = 100$
2. 成分 A: $0.9 \cdot F_{top} + 0.1 \cdot F_{bot} = 50$

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 0.9 & 0.1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} 100 \\ 50 \end{bmatrix}$$

目標: 求解 F_{top} 與 F_{bot}

NotebookLM

實務應用：Python 程式碼實作

material_balance.py

```python
import numpy as np

# 1. 定義係數矩陣 A (Coefficients)
A = np.array([[1, 1],
              [0.9, 0.1]])

# 2. 定義常數向量 b (Constants)
b = np.array([100, 50])

# 3. 求解線性方程組
x = np.linalg.solve(A, b)

# 4. 輸出結果
F_top, F_bot = x
print(f'Top Flow: {F_top:.2f}, Bottom Flow: {F_bot:.2f}')
```

Terminal Output

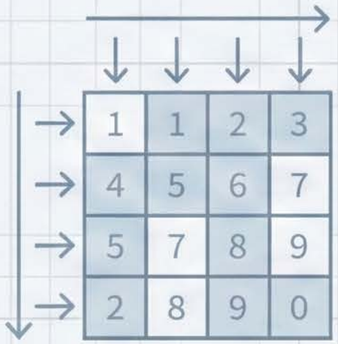
```
> Top Flow: 50.00, Bottom Flow: 50.00
```

透過簡單的程式碼，我們解決了程序模擬中最常見的平衡計算問題。

總結與最佳實踐

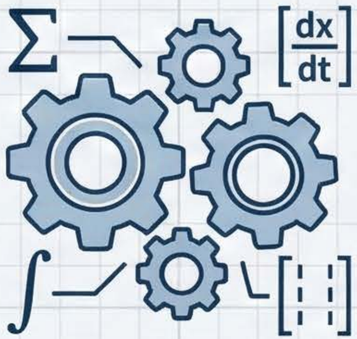
掌握 NumPy 線性代數，就是掌握了化工數值模擬的底層邏輯。

您的數位工具箱已升級



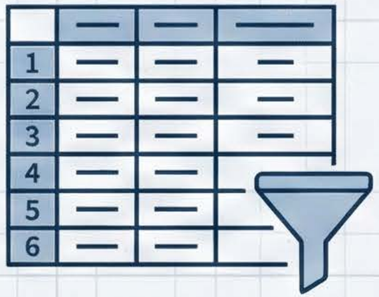
NumPy 基礎

已完成 (Completed)



線性代數引擎

Current Status: Complete



Pandas 數據分析

Next Step

下一步 (Next Step)：應用 Pandas 進行製程數據分析 (Unit 04)

現在，開啟您的 IDE，嘗試建立您的第一個物料平衡模型。

Unit 03 結束 | 前往 Unit 04

NotebookLM